



FLUTTER STARTER

Panduan Pembelajaran Lengkap Tingkat Enterprise

DILENGKAPI DENGAN 8 MODUL PEMBELAJARAN:

**Monorepo Melos • Clean Architecture • Riverpod Generator
Drift SQLite Storage • Mason CLI • Multi-Flavor & API Integration**

Penerbit: Antigravity AI Consultant

Tahun Rilis: 2026

■ Panduan Belajar Flutter Starter untuk Pemula

Selamat datang di modul panduan belajar **Flutter Starter**! Proyek ini menggunakan arsitektur modern berskala besar (*enterprise-ready*). Bagi *programmer* pemula atau yang baru transisi dari proyek Flutter skala kecil, arsitektur monorepo, arsitektur Clean Architecture, dan *state management* Riverpod modern dengan generator di proyek ini mungkin terasa asing pada awalnya.

Modul tutorial ini dirancang khusus untuk memandu Anda memahami **konsep-konsep kunci** yang ada di starter project ini agar Anda bisa berkontribusi dengan cepat dan percaya diri.

■■ Peta Jalur Belajar (Curriculum Table)

Kami membagi konsep-konsep tingkat lanjut di proyek ini menjadi beberapa bagian yang mudah dicerna:

No	Modul Belajar	Deskripsi Konsep	Tingkat Kesulitan
---	---	---	---
1	■ Monorepo & Melos	Memahami konsep multi-package, konfigurasi Melos, aturan dependensi, serta perbedaan antara <i>relative imports</i> vs <i>barrel exports</i> .	■■ (Mudah)

2	■■ Clean Architecture	Memahami pemisahan tanggung jawab 3-layer (Data, Domain, Presentation), kegunaan <i>UseCase</i> , dan hubungan antar layer.	■■■ (Sedang)
3	■ Riverpod Generator	Menguasai cara kerja anotasi @riverpod modern, <i>Notifier</i> , proses auto-generation .g.dart , serta kapan harus menggunakan ref.watch vs ref.read .	■■■■ (Cukup Menantang)
4	■■ Drift Database & Local Cache	Mengerti implementasi penyimpanan offline SQLite menggunakan Drift, skema tabel, mekanisme cache generik, dan kebijakan <i>TTL (Time-to-Live)</i> .	■■■■ (Cukup Menantang)
5	■ Mason Bricks & Automasi	Belajar cara menghemat waktu dengan generator otomatis mason untuk membuat fitur baru secara instan dalam 1 detik.	■■ (Mudah)
6	■ Barrel Exports & L10n	Mengerti pentingnya <i>barrel exports</i> untuk kebersihan import dan bagaimana sistem multi-bahasa (<i>localization</i>) terintegrasi otomatis.	■■ (Mudah)

7	■ Multi-Flavor & Environment	Memahami konsep pemisahan Flavor (Main vs Variant) dan Environment (Dev, Staging, Prod), serta cara mengompilasi dan menjalankannya.	■■■ (Sedang)
8	■ Mengimplementasi API Baru	Rangkaian langkah-demi-langkah konkrit dan lengkap membangun integrasi API REST baru dari Domain hingga Presentation Layer.	■■■■ (Cukup Menantang)

■ Mindset Penting Sebelum Memulai

Sebelum Anda masuk menulis kode, pastikan Anda memahami beberapa prinsip emas proyek ini:

1. **Anti-Manual Boilerplate:** Jangan menulis folder Clean Architecture atau Provider Riverpod secara manual dari nol. Gunakan **mason make feature** dan jalankan **dart run melos run codegen**.
2. **Ketat dengan Aturan Impor:** Kode di folder **packages/core** tidak boleh mengimpor apa pun dari **features_shared** atau **apps**. Ini demi menjaga modularitas dan kebersihan *dependency tree*.
3. **Pahami Flavor vs Environment:** Jangan pernah hardcode URL backend. Selalu gunakan konfigurasi per-flavor dan per-environment.
4. **Selalu Jalankan Tes:** Proyek ini memiliki safety net berupa unit & widget test. Sebelum melakukan *push* kode baru, pastikan semua tes lolos dengan perintah **dart run melos run test**.

Mari mulai perjalanan belajar Anda dengan menekan tautan modul pertama di bawah ini!

■ **[Mulai Modul 1: Monorepo & Melos](#)**

■ Modul 1: Memahami Monorepo & Melos

Bagi pemula, melihat proyek dengan banyak folder `pubspec.yaml` sering kali membingungkan. Biasanya, satu aplikasi Flutter berada dalam satu repositori tunggal. Proyek ini berbeda—ia menggunakan pendekatan **Monorepo**.

Mari kita bedah apa itu Monorepo, kegunaan Melos, dan aturan penting di dalamnya!

1. Apa itu Monorepo?

Monorepo (Monolithic Repository) adalah strategi pengembangan perangkat lunak di mana beberapa proyek atau *package* disimpan dalam satu repositori Git yang sama.

Di dalam proyek ini, kita memiliki:

1. **apps/main**: Aplikasi Flutter utama (misalnya aplikasi untuk pengguna umum).
2. **apps/variant**: Aplikasi varian lain (misalnya aplikasi *white-label* untuk klien tertentu).
3. **packages/core**: Fondasi kode yang digunakan bersama oleh semua aplikasi (tema, jaringan, database local, helper).
4. **packages/features_shared**: Modul-modul fitur siap pakai (seperti **auth**, **profile**, **settings**) yang bisa dipasang di aplikasi mana pun.

CATATAN

Dengan monorepo, kita tidak perlu menduplikasi kode seperti **AuthScreen** jika ingin membuat dua aplikasi berbeda. Cukup buat fiturnya di **features_shared**, lalu pasang di **apps/main** dan **apps/variant**.

2. Mengapa Menggunakan Melos?

Jika kita memiliki banyak *package*, mengelola dependensi bisa menjadi mimpi buruk. Misalnya, setiap kali kita menambah library baru di **core**, kita harus menjalankan **flutter pub get** di semua sub-folder secara manual.

Melos adalah *workspace manager* untuk monorepo Dart/Flutter. Melos bertugas:

Menghubungkan dependensi antar packages lokal (disebut bootstrapping).*

Menjalankan perintah secara massal ke seluruh package (seperti analyze, format, test, codegen).*

Perintah Melos yang Wajib Diketahui Pemula:

Perintah CLI	Apa yang Sebenarnya Terjadi di Balik Layar?
<code>---</code>	<code>---</code>
<code>dart pub get</code>	Mengunduh dependensi awal di root folder.
<code>dart run melos bootstrap</code>	Menghubungkan paket lokal (misal: membiarkan apps/main mengenali core lokal) dan menjalankan pub get di semua sub-package secara otomatis.
<code>dart run melos run codegen</code>	Menjalankan <i>build_runner</i> secara bersamaan di semua package yang membutuhkan generator kode (Riverpod, Drift, dll.).
<code>dart run melos run test</code>	Menjalankan seluruh unit & widget test di semua folder sekaligus.

3. Aturan Dependensi Arah Tunggal (Strict Dependency Rule)

Agar arsitektur monorepo tidak menjadi berantakan ("spaghetti"), proyek ini menerapkan aturan dependensi satu arah yang **sangat ketat**:

```
graph TD
    apps_main[apps/main] --> packages_core[packages/core]
    apps_main --> packages_features[packages/features_shared]
    apps_variant[apps/variant] --> packages_core
    apps_variant --> packages_features
    packages_core --> packages_features
    packages_features --> packages_core
    style packages_core fill:#4CAF50,stroke:#333,stroke-width:2px,color:#fff
    style packages_features fill:#2196F3,stroke:#333,stroke-width:2px,color:#fff
    style apps_main fill:#9C27B0,stroke:#333,stroke-width:2px,color:#fff
    style apps_variant fill:#FF9800,stroke:#333,stroke-width:2px,color:#fff
```

- **core** berada di tingkat paling bawah. **core TIDAK BOLEH** mengimpor kode apa pun dari **features_shared** maupun dari **apps**.
- **features_shared** boleh mengimpor **core**, tetapi **TIDAK BOLEH** mengimpor kode apa pun dari folder **apps**.
- **apps** berada di tingkat teratas. Aplikasi boleh mengimpor modul dari **core** dan **features_shared**.

PERINGATAN!

Melanggar aturan arah dependensi ini akan menyebabkan *circular dependency* (saling ketergantungan melingkar) yang membuat aplikasi gagal dikompilasi!

4. Konvensi Import: Relative vs Barrel Import

Salah satu kesalahan pemula yang paling sering ditemukan di monorepo adalah cara menulis **import**.

Aturan 1: Gunakan *Relative Import* di Dalam Package Sendiri

Jika Anda sedang menulis kode di dalam folder `packages/features_shared/lib/src/...`, dan ingin mengimpor berkas lain yang masih berada di dalam package `features_shared` yang sama, Anda **WAJIB** menggunakan *relative import*.

```
// ■ BENAR (Relative import) import '../domain/entities/user.dart'; import
'auth_state.dart'; // ■ SALAH (Jangan gunakan absolute package import untuk
internal package) import
'package:features_shared/src/auth/domain/entities/user.dart';
```

Aturan 2: Gunakan *Barrel Import* dari Luar Package

Jika Anda sedang menulis kode di `apps/main` dan ingin menggunakan fitur dari `features_shared` atau `core`, Anda **DILARANG** mengimpor file internal (`lib/src/...`) secara langsung. Anda **WAJIB** mengimpor via berkas pintu utama (*barrel export*).

```
// ■ BENAR (Barrel import melalui gerbang utama) import
'package:features_shared/features_shared.dart'; import
'package:core/core.dart'; // ■ SALAH (Jangan mengintip folder internal src
package lain secara langsung) import
'package:features_shared/src/auth/presentation/login_screen.dart';
```

Mengapa ini penting? Karena folder `src` dianggap sebagai area privat dari package tersebut. Pintu keluar-masuk resmi hanya melalui file barrel utama di root package (`core.dart` atau `features_shared.dart`).

Modul berikutnya akan membahas tentang bagaimana folder di dalam package fitur disusun menggunakan Clean Architecture!

■ [Lanjut ke Modul 2: Clean Architecture](#)

■■ Modul 2: Menguasai Clean Architecture

Ketika pertama kali melihat modul fitur seperti **auth** atau **profile**, programmer pemula sering bertanya: *"Mengapa kita butuh begitu banyak folder dan berkas untuk tugas sederhana seperti melakukan login?"*

Jawabannya adalah **Clean Architecture**. Pola ini memisahkan kode berdasarkan fungsinya agar mudah diuji (*testable*), mudah diganti komponennya (misal ganti REST API ke GraphQL), dan terstruktur rapi.

Mari kita pelajari 3 layer utama di starter project ini!

1. Tiga Layer Utama

Setiap fitur yang Anda buat menggunakan perintah **mason make feature** akan secara otomatis memiliki 3 folder utama: **data**, **domain**, dan **presentation**.

```
feature_name/ ■■■ data/           ← Bekerja dengan Database & API (Dunia  
Luar) ■■■ domain/             ← Logika Bisnis Utama (Aturan Murni Dart) ■■■  
presentation/ ← Tampilan UI & State Management (Widget & Notifier)
```

2. Bedah Layer Secara Mendalam

Mari kita telusuri peran masing-masing layer dengan studi kasus fitur **Login**:

A. Domain Layer (Otak Aplikasi)

Domain layer adalah bagian paling penting. Layer ini adalah **pure Dart** (tidak boleh mengimpor Flutter, UI, Dio, atau Drift). Layer ini berisi aturan bisnis inti aplikasi Anda.

1. **Entity**: Representasi objek data bisnis yang murni.

Contoh*: **User** class dengan properti **id**, **name**, dan **email**.

2. **Repository Contract (Abstract Class)**: Deklarasi fungsi-fungsi apa saja yang harus tersedia, tetapi tanpa menulis implementasi detailnya.

Contoh*: **abstract class AuthRepository** mendeklarasikan **Future login(String email, String password);**.

3. **UseCase**: Logika spesifik untuk satu aksi bisnis tertentu. Satu UseCase hanya boleh melakukan **satu tugas**.

Contoh*: **LoginUseCase** bertugas memvalidasi email/password lalu memanggil repository untuk eksekusi login.

TIPS

Mengapa menggunakan abstract class di domain? Agar jika kelak kita ingin membuat unit test, kita bisa membuat repository palsu (*MockAuthRepository*) dengan mudah tanpa perlu terhubung ke jaringan asli!

B. Data Layer (Sumber Data)

Data layer bertugas mengambil data dari dunia luar (Internet/REST API, Local database Drift, Shared Preferences) lalu menyalurkannya ke Domain Layer.

1. **DataSource**: Interface langsung dengan API atau database lokal.

* **RemoteDataSource**: Melakukan HTTP request (menggunakan Dio Client) ke server backend.

* **LocalDataSource**: Mengambil/menyimpan data lokal (menggunakan database Drift atau Secure Storage).

2. **Model**: Representasi data berformat JSON dari API/Database. Model bertugas mengonversi JSON ke objek Dart (**fromJson**) dan sebaliknya (**toJson**). Model biasanya meng-extend atau mengimplementasikan Entity dari Domain Layer.

Contoh*: **UserModel** yang meng-extend **User** entity.

3. **Repository Implementation (_impl.dart)**: Realisasi konkret dari kontrak abstract class yang dideklarasikan di Domain Layer. Repository Impl menggabungkan data dari Remote & Local DataSource.

Contoh*: **AuthRepositoryImpl** memanggil **AuthRemoteDataSource.login()**, lalu jika sukses, menyimpan token user ke **AuthLocalDataSource** (secure storage), kemudian mengembalikan objek **User**.

C. Presentation Layer (Tampilan & Kontroler)

Presentation layer berisi semua hal yang dapat dilihat oleh pengguna dan status (*state*) dari tampilan tersebut.

1. **Screen**: Halaman utama aplikasi (Widget utuh yang memiliki rute).

Contoh*: **LoginScreen**.

2. **Widget**: Komponen UI kecil yang dapat digunakan kembali di dalam screen.

Contoh*: **CustomTextField**, **LoginButton**.

3. **Notifier**: Pengatur keadaan (*state manager*) yang mengonsumsi UseCase dari domain layer dan memperbarui UI. Di proyek ini kita menggunakan **Riverpod Generator**.

Contoh*: **AuthNotifier** yang mengatur status layar dari **AuthInitial**, **AuthLoading**, hingga **AuthAuthenticated** atau **AuthError**.

3. Aliran Data (Data Flow)

Bagaimana lingkaran interaksi ini bekerja ketika pengguna menekan tombol **Login**?

```
sequenceDiagram
    actor User
    participant UI as LoginScreen
    participant Notifier as AuthNotifier (Presentation)
    participant UseCase as LoginUseCase (Domain)
    participant Repo as AuthRepositoryImpl (Data)
    participant API as AuthRemoteDataSource (Data)

    User->>UI: Input email & password, klik tombol Login
    UI->>Notifier: ref.read(authNotifierProvider.notifier).login(...)
    Notifier->>Notifier: Set State = AuthLoading() (UI loading spinner muncul)
    Notifier->>UseCase: loginUseCase(email, password)
    UseCase->>Repo: repository.login(email, password)
    Repo->>API: remoteDataSource.login(email, password)
    API-->>Repo: Mengembalikan JSON UserModel
    Repo-->>UseCase: Mengembalikan objek User
    UseCase-->>Notifier: Mengembalikan objek User
    Notifier->>Notifier: Set State = AuthAuthenticated(user)
    Notifier-->>UI: UI mendeteksi perubahan state, pindahkan halaman ke HomeScreen!
```

Meskipun terlihat panjang di awal, keuntungan arsitektur ini sangat terasa saat aplikasi bertambah besar. Setiap bagian memiliki tanggung jawab tunggal (*Single Responsibility*), membuat kode sangat mudah dibaca, didebug, dan diuji secara terpisah.

Modul berikutnya akan membahas bagaimana kita mengelola keadaan (*State Management*) menggunakan Riverpod secara modern dan otomatis!

■ [Lanjut ke Modul 3: Riverpod Generator](#)

■ Modul 3: Memahami Riverpod Generator

Bagi programmer pemula yang terbiasa dengan provider manual (seperti **StateNotifierProvider** versi lama), sintaks Riverpod di starter project ini mungkin terlihat misterius. Mengapa tidak ada deklarasi provider manual? Di mana provider itu dibuat?

Proyek ini menggunakan **Riverpod Generator** (**@riverpod**). Ini adalah standar penulisan Riverpod paling modern yang didukung resmi oleh penciptanya (Remi Rousselet).

Mari kita bedah cara kerjanya!

1. Perbedaan Gaya Penulisan: Manual vs Generator

Sebelum ada generator, kita harus menulis banyak boilerplate kode untuk satu provider:

```
// ■ GAYA LAMA (Boilerplate manual, panjang, & rentan salah tipe data)
final authNotifierProvider = StateNotifierProvider<AuthNotifier,
AuthState>((ref) {  final repo = ref.watch(authRepositoryProvider);
  return AuthNotifier(repo); });
```

Dengan **Riverpod Generator**, Anda cukup menulis kelas Notifier biasa dan memberikan anotasi **@riverpod**. Generator akan otomatis membuatkan variabel **authNotifierProvider** untuk Anda!

```
// ■ GAYA BARU (Menggunakan Generator, ringkas, dan aman)
@Riverpod(keepAlive: true) class AuthNotifier extends _AuthNotifier {
  @override  AuthState build() {  // Inisialisasi awal di sini
    return const AuthInitial();  } }
```

PENTING!

Perhatikan kata kunci `_AuthNotifier`. Kelas dasar (superclass) berawalan `_Name` ini di-generate otomatis oleh Riverpod Generator di dalam file pendamping `.g.dart`. Jika Anda melihat garis merah pada nama class ini saat membuat file baru, jangan panik! Cukup jalankan skrip codegen di terminal: `dart run melos run codegen`

2. Kapan Harus Menggunakan `@riverpod` Class vs Function?

Ada dua jenis provider yang bisa di-generate otomatis:

A. Generator Berbasis Fungsi (Functional Provider)

Gunakan jika Anda hanya butuh data read-only yang bergantung pada provider lain, atau objek instansi statis (tidak ada *logic* pengubah state di dalamnya).

Contoh: Dependency Injection untuk Repository.

```
@riverpod AuthRepository authRepository(Ref ref) {  final remoteDS =
  ref.watch(authRemoteDataSourceProvider);  final localDS =
  ref.watch(authLocalDataSourceProvider);  return
  AuthRepositoryImpl(remoteDS, localDS); }
```

Hasil generate: Otomatis membuat provider bernama `authRepositoryProvider`.

B. Generator Berbasis Kelas (Class-based Notifier)

Gunakan jika Anda membutuhkan logika untuk mengubah *state* (misalnya dari `initial` ke `loading` lalu `success`).

Contoh: Mengatur status login user.

```
@riverpod class LoginNotifier extends _LoginNotifier {  @override
  AsyncValue<void> build() {    return const AsyncValue.data(null); //
  Nilai awal state  }  Future<void> submit(String email, String password)
  async {    state = const AsyncLoading();    state = await
  AsyncValue.guard(() => ref.read(authRepositoryProvider).login(email,
  password));  } }
```

3. Aturan Emas Riverpod: `ref.watch` vs `ref.read`

Kesalahan paling umum pemula adalah salah menggunakan `ref.watch` dan `ref.read`. Ingat baik-baik aturan berikut:

`ref.watch` (Untuk Memantau Perubahan State & Rebuild UI)

- **Tempat:** Di dalam method `build(BuildContext context, WidgetRef ref)` pada widget, atau di dalam fungsi `build()` milik notifier.
- **Kegunaan:** Memantau state. Jika nilai provider berubah, widget Anda akan di-rebuild secara otomatis agar layar menampilkan data terbaru.

```
@override Widget build(BuildContext context, WidgetRef ref) {  // ■ BENAR:
Memantau perubahan status auth  final authState =
ref.watch(authNotifierProvider);  return authState.when(    data: (data)
=> Text('Welcome ${data.name}'),    loading: () =>
CircularProgressIndicator(),    error: (err, stack) => Text('Error:
${err}',    ); }
```

`ref.read` (Untuk Menjalankan Aksi / Event Callback)

Tempat: Di dalam fungsi penanganan event seperti `onPressed`, `onTap`, atau di dalam fungsi inisiasi lifecycle*.

- **Kegunaan:** Memanggil metode atau aksi dari provider **sekali jalan**. Kita tidak ingin me-rebuild tombol ketika state berubah, kita hanya ingin memicu aksi.

```
ElevatedButton(  onPressed: () {      // ■ BENAR: Menggunakan ref.read untuk memicu fungsi login      ref.read(authNotifierProvider.notifier).login(email: _emailController.text, password: _passwordController.text, );  },  child: const Text('Login'), )
```

PERINGATAN!

DILARANG KERAS menggunakan **ref.read** di dalam method **build** widget! Ini akan membuat UI tidak responsif (tidak ter-rebuild) saat data berubah, atau sebaliknya menyebabkan *bug* rendering yang tidak terprediksi.

4. Siklus Hidup Provider: **autoDispose** vs **keepAlive**

Secara bawaan (*default*), anotasi **@riverpod** akan bersifat **autoDispose**. Artinya, ketika widget yang memakai provider tersebut ditutup (tidak tampil lagi di layar), state dari provider tersebut akan dihapus dari memori secara otomatis.

Jika Anda ingin menyimpan state tersebut agar tetap ada sepanjang aplikasi berjalan (misal: data user login, tema aplikasi, lokalisasi bahasa), tambahkan parameter **keepAlive: true**:

```
// State tidak akan pernah dibuang dari memori @Riverpod(keepAlive: true) class AuthNotifier extends _AuthNotifier { ... }
```

Modul berikutnya akan menjelaskan bagaimana database lokal Drift SQLite diintegrasikan sebagai sistem caching offline!

■ [Lanjut ke Modul 4: Drift Database & Local Cache](#)

■■ Modul 4: Drift Database & Local Cache

Salah satu keunggulan utama dari starter project ini adalah dukungan **Offline-first** menggunakan database SQLite terstruktur berkinerja tinggi: **Drift** (sebelumnya dikenal sebagai Moor).

Bagi pemula yang hanya terbiasa dengan **SharedPreferences** (penyimpanan key-value sederhana), mengelola database relasional di Flutter bisa terasa rumit.

Mari kita bahas bagaimana Drift bekerja dan bagaimana sistem caching otomatis di proyek ini dibangun!

1. Mengapa Menggunakan Drift SQLite?

SharedPreferences sangat lambat dan tidak cocok untuk menyimpan data kompleks yang berstruktur besar (seperti daftar produk, riwayat transaksi, atau data profil lengkap).

Drift memberikan kita kekuatan:

- Database SQLite asli yang berkinerja tinggi.

Keamanan Tipe Data (Type-safe queries*) ditulis dengan sintaks Dart yang ramah pengembang.

- Dukungan migrasi skema database yang andal.

Di starter project ini, file konfigurasi database utama dapat ditemukan di [app_database.dart](#).

2. Mendefinisikan Tabel Database di Drift

Untuk membuat tabel baru di SQLite menggunakan Drift, kita cukup mendeklarasikan kelas Dart yang meng-extend kelas **Table**.

Mari kita lihat tabel **CacheEntries** bawaan kita:

```
class CacheEntries extends Table { // id sebagai primary key yang auto
  increment IntColumn get id => integer().autoIncrement(); // key
  bersifat unik untuk menyimpan identitas cache (misal: 'get_profile_1')
  TextColumn get key => text().unique(); // payload berformat JSON
  string hasil dari API response TextColumn get jsonPayload => text();
  // Tanggal & waktu entri cache dibuat DateTimeColumn get createdAt =>
  dateTime().withDefault(currentDateAndTime()); // Batas kedaluwarsa cache
  dalam satuan detik (null jika tidak kedaluwarsa) IntColumn get ttlSeconds
  => integer().nullable(); }
```

Setelah tabel ditulis, kita mendaftarkannya di anotasi **@DriftDatabase**:

```
@DriftDatabase(tables: [CacheEntries]) class AppDatabase extends
  _AppDatabase { ... }
```

Setelah itu, jalankan **dart run melos run codegen** agar Drift meng-generate file SQL dan pemetaan objek Dart di berkas **.g.dart**.

3. Strategi Caching Pintar dengan TTL (Time-To-Live)

Salah satu bagian tercerdas di proyek ini adalah pemanfaatan database lokal sebagai **Local Cache Generik**. Alih-alih membuat tabel untuk setiap jenis respon API, kita menyimpan respon API sebagai string JSON di dalam tabel **CacheEntries** dengan waktu kedaluwarsa (TTL).

Bagaimana mekanisme pembacaan cache bekerja? Perhatikan logika berikut di [app_database.dart](#):

```
Future<String?> readCache(String cacheKey) async { // 1. Cari data di
  database berdasarkan key unik final entry = await (select(cacheEntries)
    ..where((t) => t.key.equals(cacheKey))) .getSingleOrNull();
  if (entry == null) return null; // Data tidak ada di lokal // 2. Jika
  TTL diatur, hitung apakah cache sudah kedaluwarsa if (entry.ttlSeconds !=
  null) { final age =
    DateTime.now().difference(entry.createdAt).inSeconds; if (age >
    entry.ttlSeconds!) { // Cache sudah basi! Hapus dari database dan
    kembalikan null await deleteCache(cacheKey); return null; }
  } // 3. Cache masih segar, kembalikan isi payload-nya! return
  entry.jsonPayload; }
```

4. Mekanisme Pembersihan Otomatis (Eviction Policy)

Untuk mencegah memori perangkat pengguna penuh oleh data cache yang sudah kedaluwarsa (*stale cache*), kita memiliki fungsi **evictExpired()**. Fungsi ini secara asinkron memindai seluruh tabel dan menghapus entri cache yang masa berlakunya telah habis.

```
Future<int> evictExpired() async { final now = DateTime.now(); final
  all = await select(cacheEntries).get(); int deleted = 0; for (final
  entry in all) { if (entry.ttlSeconds != null) { final age =
  now.difference(entry.createdAt).inSeconds; if (age >
  entry.ttlSeconds!) { await (delete(cacheEntries)..where((t) =>
  t.id.equals(entry.id))).go(); deleted++; } } } return
  deleted; }
```

Bagaimana Cara Programmer Pemula Menggunakan Sistem Cache Ini?

Di layer Repository Implementation, Anda cukup memanfaatkan provider **appDatabaseProvider** untuk menulis dan membaca cache sebelum melakukan *network request* ke internet.

Contoh sederhana:

```
Future<Profile> getProfile(String userId) async {  final cacheKey =
'profile_$userId';      // 1. Coba baca dari local database dahulu  final
cachedJson = await _database.readCache(cacheKey);  if (cachedJson != null)
{      return ProfileModel.fromJson(jsonDecode(cachedJson));  }      // 2.
Jika cache kosong/expired, panggil API internet  final freshModel = await
_remoteDataSource.getProfile(userId);      // 3. Simpan data baru tersebut
ke cache untuk pemanggilan berikutnya dengan TTL 1 Jam (3600 detik)  await
_database.writeCache(cacheKey, jsonEncode(freshModel.toJson()), ttlSeconds:
3600);      return freshModel; }
```

Sistem caching ini sangat ampuh dalam mengurangi beban server backend, menghemat kuota internet pengguna, dan membuat transisi aplikasi terasa instan bagi pengguna!

Modul berikutnya akan memandu Anda mempercepat proses coding modul baru menggunakan kekuatan Mason!

■ [Lanjut ke Modul 5: Mason Bricks & Automasi](#)

■ Modul 5: Mason Bricks & Automasi Pembuatan Fitur

Salah satu hambatan terbesar bagi pengembang ketika bekerja dengan arsitektur berskala besar seperti Clean Architecture adalah **kejenuhan menulis kode boilerplate**.

Untuk membuat satu fitur sederhana seperti "fitur kupon", Anda harus membuat 10+ folder dan berkas kosong, menulis import yang tepat, membuat kerangka class untuk DataSource, Repository, Notifier, Screen, hingga State. Hal ini membuang waktu dan sangat rentan terhadap kesalahan ketik (*typo*).

Di proyek ini, kita memecahkan masalah tersebut dengan **Mason CLI**.

1. Apa itu Mason?

Mason adalah alat pembuat template kode (*code generator*) yang andal untuk Flutter/Dart. Mason menggunakan konsep **Bricks** (cetakan bata) yang dapat kita rakit dan jalankan lewat baris perintah (CLI).

Proyek kita mendefinisikan template modular di folder [bricks/feature](#). Cetakan ini sudah dirancang agar menghasilkan berkas Clean Architecture yang 100% pas dengan konvensi proyek ini, termasuk:

- Menggunakan Riverpod generator modern.
- Membuat struktur folder 3-layer (**data**, **domain**, **presentation**).
- Menyediakan nama kelas dinamis berdasarkan input (misal **PaymentScreen**, **PaymentNotifier**).

Mengatur relative imports* secara otomatis dan presisi.

2. Cara Menggunakan Mason di Proyek Ini

Kami telah menyederhanakan pemanggilan Mason melalui integrasi perintah Melos. Anda tidak perlu menginstal Mason secara global di komputer Anda!

Langkah 1: Jalankan Perintah Pembuatan Fitur

Buka terminal Anda di root folder proyek ini, dan jalankan perintah sakti berikut:

```
dart run melos run make-feature
```

Langkah 2: Isi Pertanyaan Interaktif

Terminal akan menampilkan prompt interaktif:

1. What is the feature name?

Masukkan nama fitur Anda menggunakan huruf kecil (lowercase*), misalnya: **payment** atau **order_history**.

2. Which package should contain this feature?

* Tekan Enter untuk memilih opsi bawaan (**features_shared**), atau arahkan ke **apps/main** jika fitur tersebut khusus hanya untuk aplikasi utama.

Langkah 3: Saksikan Keajaiban Terjadi!

Hanya dalam 1 detik, Mason akan mencetak struktur folder lengkap di dalam [packages/features_shared/lib/src/payment](#).

Semua kelas pendukung seperti `PaymentLocalDataSource`, `PaymentRepositoryImpl`, `PaymentUseCase`, `PaymentNotifier`, hingga `PaymentScreen` telah siap digunakan!

3. Apa yang Harus Dilakukan Setelah Menjalankan Mason?

Setelah file-file dasar terbuat secara otomatis, ada **dua langkah wajib** yang harus Anda lakukan agar kode baru tersebut dapat dikenali dan dikompilasi tanpa error:

Wajib 1: Jalankan Code Generation (Codegen)

Karena cetakan Mason menyertakan notifier berbasis Riverpod generator, Anda akan melihat garis merah error pada kelas dasar `_$PaymentNotifier` yang belum di-generate.

Jalankan perintah ini di root terminal untuk menghasilkan berkas `.g.dart` yang hilang:

```
dart run melos run codegen
```

Wajib 2: Daftarkan ke Pintu Gerbang (Barrel Export)

Ingat aturan monorepo di Modul 1? Aplikasi luar tidak boleh mengintip folder internal `src/` secara langsung.

Anda harus mendaftarkan kelas-kelas yang ingin diakses oleh aplikasi luar ke berkas barrel export utama di [packages/features_shared/lib/features_shared.dart](#).

Cukup tambahkan baris ekspor di akhir berkas tersebut, misalnya:


```
// payment – presentation export  
'src/payment/presentation/payment_screen.dart'; export  
'src/payment/presentation/payment_provider.dart';
```

4. Aturan Wajib AI Agent: Penggunaan Mason

PENTING!

Proyek ini memiliki aturan ketat untuk seluruh developer maupun **AI Agent** yang ditugaskan bekerja di repositori ini: **Setiap kali menambahkan modul/fitur baru, WAJIB menggunakan perintah `mason make feature` (atau `melos run make-feature`).** Dilarang keras menulis folder arsitektur secara manual demi menjaga konsistensi kode 100%!

Modul terakhir akan menjelaskan mengenai pentingnya gerbang *Barrel Exports* dan bagaimana sistem multi-bahasa (Localization) diatur secara terpusat!

■ [Lanjut ke Modul 6: Barrel Exports & L10n](#)

■ Modul 6: Barrel Exports & Localization (L10n)

Selamat! Anda telah sampai di modul terakhir dari panduan belajar Flutter Starter. Di modul ini, kita akan membahas dua detail kecil namun sangat berdampak pada kerapian dan fungsionalitas aplikasi: **Barrel Exports** dan **Sistem Multi-Bahasa (Localization / L10n)**.

1. Apa itu Barrel Exports?

Pernahkah Anda melihat berkas impor di Flutter yang sangat panjang dan berantakan seperti ini?

```
// ■ SANGAT BERANTAKAN (Banyak baris impor untuk satu fitur) import
'package:features_shared/src/auth/domain/entities/user.dart'; import
'package:features_shared/src/auth/domain/usecases/login_use_case.dart';
import 'package:features_shared/src/auth/presentation/login_screen.dart';
import 'package:features_shared/src/auth/presentation/auth_state.dart';
```

Untuk mengatasinya, kita menggunakan pola **Barrel Export**.

Konsepnya sederhana: Kita membuat satu berkas "pintu masuk utama" di root package yang bertugas mengekspor semua file internal penting dari sub-folder **src**.

Di proyek ini, berkas pintu gerbang tersebut berada di:

- [packages/core/lib/core.dart](#)
- [packages/features_shared/lib/features_shared.dart](#)

Di dalam file barrel tersebut, kita menulis deklarasi ekspor seperti:

```
export 'src/auth/domain/entities/user.dart'; export
'src/auth/presentation/login_screen.dart';
```

Berkat adanya barrel export ini, siapapun (termasuk modul aplikasi utama di **apps/main**) cukup menulis **satu baris impor** saja untuk mengakses seluruh fitur tersebut!

```
// ■ SANGAT RAPI (Cukup satu baris impor) import
'package:features_shared/features_shared.dart';
```

PERINGATAN!

Ingat aturan penting ini! Setiap kali Anda membuat berkas baru (misal: UseCase baru, model baru, widget baru) di dalam package, pastikan Anda mendaftarkan ekspor berkas tersebut di file barrel utama (**core.dart** atau **features_shared.dart**) agar bisa dibaca dari luar package. Jika Anda lupa, compiler akan menganggap berkas tersebut bersifat privat dan aplikasi luar tidak akan bisa memanggilnya.

2. Bagaimana Sistem Multi-Bahasa (Localization / L10n) Bekerja?

Di starter project ini, sistem multibahasa tidak lagi ditulis secara hardcoded di dalam widget, melainkan dipusatkan di dalam paket **core** menggunakan modul **l10n** bawaan Flutter (**flutter_localizations**).

A. Di mana Letak Berkas Terjemahan?

Semua berkas kamus kata bahasa disimpan dalam format **.arb** (Application Resource Bundle) di folder [packages/core/lib/src/l10n](#):

- **app_id.arb**: Berisi kamus terjemahan Bahasa Indonesia.
- **app_en.arb**: Berisi kamus terjemahan Bahasa Inggris.

Format penulisannya berbentuk key-value JSON seperti:

```
{  "login": "Masuk",    "email": "Surel",    "password": "Kata Sandi" }
```

B. Bagaimana Cara Meng-generate File L10n?

Setiap kali Anda menambah key baru di file **.arb**, Anda harus men-generate file kode Dart-nya agar bisa dipanggil dengan aman (*type-safe*).

Jalankan perintah pintas Melos berikut di terminal Anda:

```
dart run melos run l10n
```

C. Cara Menggunakannya di UI (Widget)

Gunakan ekstensi bawaan **BuildContext** dari core untuk memanggil kata yang sudah diterjemahkan:

```
import 'package:core/core.dart'; // Impor core terlebih dahulu class
MyWidget extends StatelessWidget { @override Widget build(BuildContext
context) {    return Column(        children: [            // ■ Memanggil
terjemahan teks "login" secara aman        Text(context.l10n.login),
            // ■ Memanggil terjemahan teks "password"
            Text(context.l10n.password),        ],    ); } }
```

Sistem akan secara otomatis mendeteksi bahasa perangkat pengguna saat ini (atau bahasa yang dipilih pengguna di menu pengaturan) dan mengganti teks tersebut secara dinamis tanpa restart aplikasi!

■ Langkah Terakhir

Modul berikutnya akan menjelaskan bagaimana aplikasi Utama (main) dan Varian dipisahkan menggunakan Flavor dan bagaimana Environment dikonfigurasi!

■ [Lanjut ke Modul 7: Multi-Flavor & Environment](#)

Modul 7: Memahami Multi-Flavor & Multi-Environment

Bagi pemula, istilah **"Flavor"** dan **"Environment"** sering kali dianggap sama. Namun, di starter project kelas dunia ini, keduanya dipisahkan secara bersih untuk mendukung skenario bisnis **SaaS (Software-as-a-Service) / Multi-Tenant / White-Label**.

Mari kita bedah perbedaan keduanya, bagaimana kode proyek memprosesnya, dan cara menjalankannya dengan mudah!

1. Perbedaan Mendasar: Flavor vs Environment

Untuk memahaminya dengan mudah, bayangkan tabel berikut:

Konsep	Pertanyaan yang Dijawab	Contoh di Proyek Ini	Lokasi Fisik Folder
---	---	---	---
Flavor (App)	"Aplikasi milik siapa yang sedang di-build?" (Identitas, nama aplikasi, logo, konfigurasi warna)	<code>main</code> (Aplikasi utama) <code>variant</code> (Aplikasi white-label klien B)	<code>apps/main/</code> <code>apps/variant/</code>
Environment	"Server database mana yang sedang dihubungkan?" (Base URL API, debug logs, kunci API dev/prod)	<code>dev</code> (Local/Sandbox) <code>staging</code> (Uji coba QA) <code>prod</code> (Server asli konsumen)	<code>packages/core/lib/src/config/</code>

TIPS

Kombinasi Tak Terbatas: Dengan pemisahan ini, Anda bisa menjalankan aplikasi **Main** yang terhubung ke server **Staging**, atau menjalankan aplikasi **Variant** (klien white-label) yang terhubung ke server **Production**.

2. Bagaimana Environment Dibaca oleh Flutter?

Kita menghindari cara kuno seperti membuat file config terpisah yang di-load saat runtime (lambat dan rentan). Sebagai gantinya, kita menggunakan fitur bawaan Flutter: **--dart-define**.

Alur Pembacaan Environment:

1. Saat menjalankan aplikasi, kita mengirim parameter lingkungan di CLI:

```
flutter run --dart-define=ENV=dev
```

2. Di dalam kode, kelas **AppConfig** di **packages/core** bertugas mendeteksi nilai **--dart-define** tersebut:

```
enum Environment { dev, staging, prod; static
Environment fromString(String value) { return values.firstWhere(
    (e) => e.name == value.toLowerCase(),      orElse: () =>
Environment.dev,                               ); } }
```

3. Masing-masing aplikasi (**apps/main** & **apps/variant**) memiliki kelas konfigurasi konkrit (**MainConfig** dan **VariantConfig**) yang mewarisi **AppConfig** dan menentukan URL API yang dinamis:

```
// apps/main/lib/config/main_config.dart    class MainConfig extends
AppConfig {      @override      String get baseUrl => switch (environment)
{      Environment.dev => 'https://api-dev.mainproduct.com',
Environment.staging => 'https://api-staging.mainproduct.com',
Environment.prod => 'https://api.mainproduct.com',      };    }
```

3. Cara Menjalankan Kombinasi Flavor & Environment

Sebagai developer, Anda tidak perlu mengetik perintah yang panjang dan rumit di terminal setiap hari. Melos dan VS Code sudah dikonfigurasi untuk memudahkan hidup Anda!

Opsi A: Menggunakan CLI Melos (Root Folder)

Melos menyediakan pintasan (*scripts*) di **pubspec.yaml** untuk menjalankan versi *Development*:

```
# Menjalankan aplikasi Utama (main app) di lingkungan dev dart run melos
run dev # Menjalankan aplikasi Varian (variant app) di lingkungan dev dart
run melos run dev:variant
```

Opsi B: Menggunakan Launch Configurations VS Code (Sangat Direkomendasikan)

Buka menu **Run & Debug (Ctrl+Shift+D)** di VS Code. Anda akan melihat daftar konfigurasi siap pakai:

- **main (dev)** — Menjalankan aplikasi utama dengan server sandbox.
- **main (staging)** — Menjalankan aplikasi utama untuk verifikasi QA.

- **main (prod)** — Menjalankan aplikasi utama versi live.
- **variant (dev)** — Menjalankan aplikasi klien B dengan server sandbox.
- **variant (staging)** — Menjalankan aplikasi klien B versi QA.
- **variant (prod)** — Menjalankan aplikasi klien B versi live.

Cukup klik tombol **Play (F5)** pada pilihan yang diinginkan untuk memulai debug instan!

4. Keamanan Firebase Per Flavor & Environment

Karena Firebase memerlukan file konfigurasi native (**google-services.json** untuk Android dan **GoogleService-Info.plist** untuk iOS), starter project ini menggunakan teknik **Multi-Folder Configuration** untuk memisahkan Firebase di tingkat native:

```
apps/main/android/app/src/ ■■■ dev/ ■ ■■■ google-services.json      #
Firebase untuk Main Dev ■■■ staging/ ■ ■■■ google-services.json      #
Firebase untuk Main Staging ■■■ release/ ■■■
google-services.json          # Firebase untuk Main Production
```

Gradle Android dan Xcode iOS telah dikonfigurasi secara otomatis untuk menyalin file kustom yang sesuai berdasarkan flavor & environment yang sedang dikompilasi saat proses build berjalan.

CATATAN

Panduan konfigurasi konsol Firebase dan native secara lengkap dapat dibaca di berkas panduan khusus [docs/firebase-setup.md](https://docs.antigravityide.com/docs/firebase-setup.md).

■ Langkah Penutup

Modul berikutnya akan menjelaskan secara lengkap dan detail langkah-demi-langkah mengimplementasikan integrasi API REST baru dari awal hingga UI!

■ [Lanjut ke Modul 8: Mengimplementasikan API Baru](#)

Modul 8: Langkah-Demi-Langkah Mengimplementasikan API Baru

Salah satu pertanyaan terbesar bagi programmer pemula saat menggunakan arsitektur Clean Architecture adalah: **"Bagaimana cara mengimplementasikan satu API baru secara lengkap dari awal?"**

Jika kita hanya melihat potongan kode saja, aliran arsitektur ini akan terasa membingungkan. Di modul ini, kita akan melakukan simulasi praktis langkah-demi-langkah menambahkan fitur API baru: **"Get Weather / Mengambil Data Cuaca"** secara utuh dari Domain hingga Presentation!

Peta Folder Studi Kasus (Fitur Weather)

Ketika kita ingin membuat fitur baru bernama **weather**, strukturnya di dalam package **features_shared** akan seperti ini:

```
features_shared/lib/src/weather/ ■■■ domain/ ■ ■■■ entities/ ■ ■  
■■■ weather.dart # 1. Objek data bisnis murni ■ ■■■  
repositories/ ■ ■ ■■■ weather_repository.dart # 2. Kontrak  
(Interface) Repository ■ ■■■ usecases/ ■ ■■■  
get_weather_usecase.dart # 6. Logika bisnis UseCase ■■■ data/ ■ ■■■  
models/ ■ ■ ■■■ weather_model.dart # 3. Model serialisasi JSON  
API ■ ■■■ datasources/ ■ ■ ■■■ weather_remote_ds.dart # 4.  
Request API dengan Dio ■ ■■■ repositories/ ■ ■■■  
weather_repo_impl.dart # 5. Realisasi nyata kontrak repository ■■■  
presentation/ ■■■ weather_notifier.dart # 7. Riverpod State  
Notifier ■■■ weather_screen.dart # 8. Widget UI Tampilan
```

Mari kita bangun satu per satu berkas di atas!

Langkah 1: Mendefinisikan Entity (Domain Layer)

Lokasi: [domain/entities/weather.dart](#)

Entity adalah kelas Dart murni yang merepresentasikan data bisnis murni. Bagian ini bebas dari library eksternal Flutter atau HTTP.

```
// domain/entities/weather.dart class Weather {  const Weather({
  required this.cityName,    required this.temperature,    required
this.condition,    required this.humidity,  });  final String cityName;
  final double temperature;  final String condition;  final int humidity;
}
```

Langkah 2: Membuat Kontrak Repository (Domain Layer)

Lokasi: [domain/repositories/weather_repository.dart](#)

Kontrak berupa **abstract class** yang mendefinisikan apa saja fungsi yang wajib disediakan untuk mengambil data cuaca, tanpa memedulikan detail teknis (apakah lewat REST API, GraphQL, atau database lokal).

```
// domain/repositories/weather_repository.dart import
'../entities/weather.dart'; abstract class WeatherRepository {
  Future<Weather> getWeather(String city); }
```

Langkah 3: Membuat Model DTO (Data Layer)

Lokasi: [data/models/weather_model.dart](#)

Model bertugas menjembatani format JSON mentah dari API backend dengan Entity murni di Domain Layer. Model mewarisi (*extends*) atau mengimplementasikan Entity.

```
// data/models/weather_model.dart import
'../../domain/entities/weather.dart'; class WeatherModel extends Weather {
  const WeatherModel({required super.cityName, required
super.temperature, required super.condition, required
super.humidity, }); /// Mengubah JSON Map dari API server menjadi objek
WeatherModel factory WeatherModel.fromJson(Map<String, dynamic> json) {
  return WeatherModel(      cityName: json['city_name'] as String? ??
'',      // Menjaga agar parsing double aman dari int      temperature:
(json['temperature'] as num? ?? 0.0).toDouble(),      condition:
json['condition'] as String? ?? 'Clear',      humidity: json['humidity']
as int? ?? 0,      ); } /// Mengonversi objek ke JSON jika ingin
mengirim data ke server Map<String, dynamic> toJson() { return {
  'city_name': cityName,      'temperature': temperature,
  'condition': condition,      'humidity': humidity,    }; } }
```

Langkah 4: Menghubungkan API dengan Dio (Data Layer)

Lokasi: [data/datasources/weather_remote_ds.dart](#)

Di sinilah REST API dipanggil menggunakan Dio client bawaan proyek. Kita tidak melakukan penanganan error yang kompleks di sini; biarkan interceptor Dio global di [core](#) menangani logging, otorisasi token, dan mapping HTTP Exception.

```
// data/datasources/weather_remote_ds.dart import 'package:dio/dio.dart';
import '../models/weather_model.dart'; class WeatherRemoteDataSource {
  const WeatherRemoteDataSource(this._dio); final Dio _dio; ///
Memanggil endpoint: GET /v1/weather?city={city} Future<WeatherModel>
fetchWeather(String city) async { final response = await _dio.get(
  'v1/weather',      queryParameters: {'city': city},      );
  // Asumsi bentuk response: { "success": true, "data": { "city_name":
"...", ... } } final jsonMap = response.data['data'] as Map<String,
dynamic>; return WeatherModel.fromJson(jsonMap); } }
```

Langkah 5: Mengimplementasikan Repository Contract (Data Layer)

Lokasi: [data/repositories/weather_repo_impl.dart](#)

Di layer ini, kita merealisasikan kontrak abstract class **WeatherRepository** yang telah dibuat di Domain Layer. Repository Impl bertugas memanggil Remote DataSource, melakukan konversi model ke entity, dan mengelola cache jika diperlukan.

```
// data/repositories/weather_repo_impl.dart import
'../../domain/entities/weather.dart'; import
'../../domain/repositories/weather_repository.dart'; import
'../datasources/weather_remote_ds.dart'; class WeatherRepositoryImpl
implements WeatherRepository {  const
WeatherRepositoryImpl(this._remoteDataSource);  final
WeatherRemoteDataSource _remoteDataSource;  @override Future<Weather>
getWeather(String city) async {    // Memanggil API remote, mengembalikan
Weather (karena WeatherModel extends Weather)    return await
_remoteDataSource.fetchWeather(city);  } }
```

Langkah 6: Membuat UseCase (Domain Layer)

Lokasi: [domain/usecases/get_weather_usecase.dart](#)

UseCase mengonsumsi Repository murni untuk mengeksekusi satu aksi bisnis yang spesifik. Gunakan *callable class* **call()** agar UseCase bisa dipanggil seperti fungsi biasa.

```
// domain/usecases/get_weather_usecase.dart import
'../entities/weather.dart'; import
'../repositories/weather_repository.dart'; class GetWeatherUseCase {
  const GetWeatherUseCase(this._repository);  final WeatherRepository
_repository;  Future<Weather> call(String city) async {    // Melakukan
validasi tambahan jika perlu (misal: nama kota tidak boleh kosong)    if
(city.trim().isEmpty) {      throw Exception('Nama kota tidak boleh
kosong!');    }    return await _repository.getWeather(city);  } }
```

Langkah 7: Menyusun State & Notifier (Presentation Layer)

Lokasi: [presentation/weather_notifier.dart](#)

Di sini kita mendaftarkan Dependency Injection (DI) untuk DataSource, Repository, UseCase, serta membuat Notifier State menggunakan kekuatan **Riverpod Generator**.

Bagian A: Mendaftarkan DI via Provider Generator

```
// presentation/weather_providers.dart (atau digabung di notifier) import
'package:core/core.dart'; import
'package:flutter_riverpod/flutter_riverpod.dart'; import
'package:riverpod_annotation/riverpod_annotation.dart'; import
'../data/datasources/weather_remote_ds.dart'; import
'../data/repositories/weather_repo_impl.dart'; import
'../domain/repositories/weather_repository.dart'; import
'../domain/usecases/get_weather_usecase.dart'; part
'weather_providers.g.dart'; // Akan di-generate otomatis @riverpod
WeatherRemoteDataSource weatherRemoteDS(Ref ref) { // Mengambil instance
Dio global dari core package final dio =
ref.watch(dioClientProvider).dio; return WeatherRemoteDataSource(dio); }
@riverpod WeatherRepository weatherRepository(Ref ref) { final remoteDS =
ref.watch(weatherRemoteDSProvider); return
WeatherRepositoryImpl(remoteDS); } @riverpod GetWeatherUseCase
getWeatherUseCase(Ref ref) { final repository =
ref.watch(weatherRepositoryProvider); return
GetWeatherUseCase(repository); }
```

Bagian B: Membuat Notifier Menggunakan Riverpod @riverpod

```
// presentation/weather_notifier.dart import
'package:riverpod_annotation/riverpod_annotation.dart'; import
'../domain/entities/weather.dart'; import 'weather_providers.dart'; part
'weather_notifier.g.dart'; // Akan di-generate otomatis @riverpod class
WeatherNotifier extends _$WeatherNotifier { @override
  AsyncValue<Weather?> build() { // Nilai awal: Belum ada data cuaca
yang dicari return const AsyncValue.data(null); } Future<void>
fetchWeather(String city) async { state = const AsyncLoading();
// AsyncValue.guard menangani try-catch otomatis dan memetakan error ke
UI state = await AsyncValue.guard(() async { final useCase =
ref.read(getWeatherUseCaseProvider); return await useCase(city);
}); } }
```

Jangan lupa jalankan perintah generator:

```
dart run melos run codegen
```

Langkah 8: Barrel Export (Menghubungkan ke Pintu Utama)

Lokasi: [packages/features_shared/lib/features_shared.dart](#)

Daftarkan berkas layar UI dan Notifier Anda di file barrel utama agar aplikasi utama ([apps/main](#)) bisa mengimpornya dengan satu baris rapi.

```
// Tambahkan baris ekspor berikut di features_shared.dart export
'src/weather/presentation/weather_notifier.dart'; export
'src/weather/presentation/weather_screen.dart';
```

Langkah 9: Memanggil di Tampilan UI (Presentation Layer)

Lokasi: [presentation/weather_screen.dart](#)

Gunakan **ConsumerWidget** untuk mengonsumsi state dan me-rebuild UI secara responsif.

```
// presentation/weather_screen.dart import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart'; import
'package:core/core.dart'; import 'weather_notifier.dart'; class
WeatherScreen extends ConsumerStatefulWidget { const
WeatherScreen({super.key}); @override ConsumerState<WeatherScreen>
createState() => _WeatherScreenState(); } class _WeatherScreenState extends
ConsumerState<WeatherScreen> { final _cityController =
TextEditingController(); @override Widget build(BuildContext context) {
  // 1. Memantau state cuaca saat ini (Loading/Error/Data) final
  weatherState = ref.watch(weatherNotifierProvider); final theme =
  Theme.of(context); return Scaffold( appBar: AppBar(title: const
  Text('Weather Search')), body: Padding( padding: const
  EdgeInsets.all(24.0), child: Column( children: [
    TextField( controller: _cityController,
    decoration: const InputDecoration(labelText: 'Masukkan Nama
    Kota'), ), const SizedBox(height: 16),
    ElevatedButton( onPressed: () {
      // 2. Memicu aksi pencarian cuaca ref.read(
      weatherNotifierProvider.notifier).fetchWeather(_cityController.text);
    }, child: const Text('Cari Cuaca'),
    ), const SizedBox(height: 32),
    // 3. Menampilkan status UI berdasarkan State
    weatherState.when( data: (weather) {
      if (weather == null) { return const
      Text('Silakan cari kota terlebih dahulu.')}
      return Column( children: [
        Text(weather.cityName, style:
        theme.textTheme.headlineMedium),
        Text('${weather.temperature}°C', style:
        theme.textTheme.headlineLarge), Text(weather.condition,
        style: theme.textTheme.titleMedium), Text('Kelembaban:
        ${weather.humidity}%'), ], );
      ], loading: () => const
      CircularProgressIndicator(), error: (err, _) => Text(
      'Terjadi Kesalahan: $err', style: const
      TextStyle(color: Colors.red), ), ), ); } }
```

■ Kesimpulan Aliran Implementasi API

Setiap kali Anda diminta menambahkan API baru di masa depan, ikuti **9 Langkah Emas** ini secara berurutan. Ini adalah standar baku industri untuk pengembangan aplikasi berskala besar. Kode Anda akan selalu terstruktur, modular, sangat mudah diuji (*highly testable*), dan bebas dari *spaghetti code*!

■ [Kembali ke Modul 0: Peta Belajar](#)

Bila Anda ingin langsung mempraktikkan konsep 9 Langkah Emas ini pada studi kasus nyata berskala penuh (menyinkronkan state, caching database Drift lokal, dan memanggil server REST API secara riil), ikuti modul praktek di bawah ini:

■ [Praktek Nyata: Tutorial Membuat Fitur Quotes \(Clean Architecture & Drift Caching\)](#)